

UCab — Full-Stack MERN Cab Booking Platform

Solo Developer / Team Member: Shaik Sumiya Zainab | Project ID: N/A (Solo Track)

Phase 5: Project Development Phase — Code Layout, Readability & Reusability Standards

Project Name: Cab Booking (`UCab`)

Project ID: `N/A (Solo Track Submission)`

Team Member / Solo Developer: Shaik Sumiya Zainab

1. Modular Code Organization & Layout Strategy

To ensure that the UCab codebase remains maintainable, scalable, and easy to grade, strict separation of concerns (`SoC`) was enforced across both the React frontend and the Express backend.

```
5. Project Development Phase/UCab/
  % % % client/src/
  %   % % % components/           # Reusable structural widgets (`Navbar`, `Footer`,
`ReceiptModal`)
  %   % % % context/             # `AuthContext.jsx` – Global JWT session and toast notification
provider
  %   % % % pages/               # 11 distinct feature-focused view controllers (`BookCab`,
`AdminCars`, etc.)
  %   % % % index.css            # Centralized vanilla CSS design tokens, utility classes, and
keyframes
  % % % server/
  % % % controllers/             # Pure business logic separated from HTTP route definitions
  % % % db/                     # Dual-engine database adapters (`config.js` and `store.js`)
  % % % middleware/             # Security validation (`authMiddleware.js`) and file uploaders
(`multer.js`)
  % % % models/                 # Mongoose schemas (`User.js`, `Car.js`, `Booking.js`, `Admin.js`)
  % % % routes/                 # Clean REST route definitions linking HTTP verbs to controllers
```

2. Readability Best Practices Applied

2.1 Explicit Naming Conventions

- Components & Files: PascalCase (`CabListing.jsx`, `ReceiptModal.jsx`).
- Controllers & Utilities: camelCase (`carController.js`, `authMiddleware.js`).
- Environment Variables: UPPERCASE (`JWT\_SECRET`, `PORT`).

2.2 Async/Await with Robust Error Bounds

Every asynchronous API call across the frontend (`Axios`) and backend (`Mongoose / JSON Store`) is wrapped inside clean `try / catch` blocks. If an exception occurs, a clear descriptive JSON payload (`{ message: error.message }`) is returned and displayed to the user via floating toast notifications (`Toast.jsx`).

3. Component & Logic Reusability Patterns

3.1 Reusable UI Wrappers (`ProtectedUserRoute.jsx` & `ProtectedAdminRoute.jsx`)

Instead of checking `localStorage` and role permissions inside every single page component, two higher-order route wrappers were created. They inspect `userRole` inside `AuthContext` and either render the protected child element (`<Outlet />`) or automatically redirect unauthorized traffic:

```
// Example of ProtectedAdminRoute.jsx reusability:
const ProtectedAdminRoute = () => {
  const { user, userRole, loading } = useContext(AuthContext);
  if (loading) return <div className="loader">Loading Session...</div>;
  return (user && userRole === 'admin') ? <Outlet /> : <Navigate to="/admin/login"
```

```
replace />;  
};
```

3.2 Dual-Engine Storage Adapter (`server/db/store.js`)

The `store.js` engine encapsulates all atomic file read and write operations inside two pure helper functions (`loadData()` and `saveData()`). Every backend controller (`carController`, `bookingController`, `adminController`, `UserController`) reuses these unified adapters when `global.isMongoConnected` is false, eliminating code duplication across our 15+ REST API endpoints.